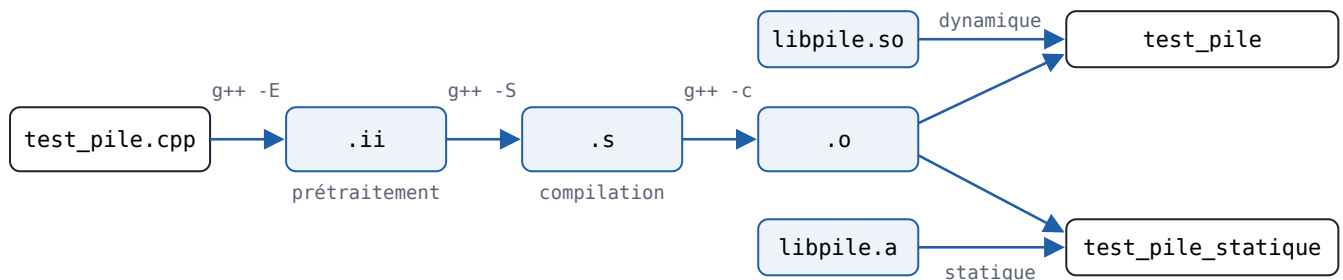


Les étapes de la compilation C++ sous Linux

TP 1 — Pile générique et bibliothèque `libpile.so` · Programmation Orientée Objet Avancée et Design Patterns

Fichiers produits par `make etapes` dans le dossier `tp1/compilation/` (g++ 15.2, Linux x86-64)

Une commande comme `g++ test_pile.cpp -o test_pile` enchaîne en réalité quatre programmes : le **préprocesseur**, le **compilateur**, l'**assembleur** et l'**éditeur de liens**. La cible `make etapes` du Makefile les exécute une par une sur `src/test_pile.cpp` et conserve le fichier produit à chaque étape. Ce document explique chacun de ces fichiers.



Fichier	Étape	Commande	Taille	Nature
<code>test_pile.ii</code>	1. prétraitement	<code>g++ -E</code>	1,1 Mo	texte C++ (42 160 lignes)
<code>test_pile.s</code>	2. compilation	<code>g++ -S</code>	49 Ko	texte, assembleur x86-64
<code>test_pile.o</code>	3. assemblage	<code>g++ -c</code>	30 Ko	binaire ELF « relocatable »
<code>libpile.a</code>	bibliothèque statique	<code>ar rcs</code>	335 Ko	archive de 3 fichiers <code>.o</code>
<code>libpile.so</code>	bibliothèque dynamique	<code>g++ -shared</code>	215 Ko	binaire ELF « shared object »
<code>test_pile</code>	4. édition de liens	<code>g++ -lpile</code>	29 Ko	exécutable, charge <code>libpile.so</code>
<code>test_pile_statique</code>	4. édition de liens	<code>g++ ... libpile.a</code>	59 Ko	exécutable autonome

À retenir. Le `.ii` et le `.s` sont des fichiers texte ; le `.o`, le `.so` et les exécutables sont des fichiers binaires ELF (vérifiable avec `file compilation/*`). Le `.o` contient du code machine, mais seule l'édition de liens, qui résout ses symboles non définis, produit un programme qui peut s'exécuter. Tous ces fichiers se régénèrent avec `make etapes`.

1. Prétraitement — `test_pile.ii`

```
g++ -Wall -Wextra -O2 -Iinclude -E src/test_pile.cpp -o compilation/test_pile.ii
```

Le préprocesseur travaille sur le **texte** du programme, sans rien comprendre au C++. Il traite toutes les lignes qui commencent par `#` :

- `#include` est remplacé par le contenu complet du fichier inclus : `<iostream>`, `<chrono>`, `<string>`, puis `PileListe.h`, `PileTableau.h` et `PileTableau.tpp` (cherchés dans `include/` grâce à `-Iinclude`).

```
Iinclude);
```

- les macros (`#define`) sont remplacées par leur valeur et les blocs `#ifdef` non retenus sont supprimés ;
- les commentaires disparaissent.

Les 114 lignes de `test_pile.cpp` deviennent ainsi **42 160 lignes**, presque toutes issues de la bibliothèque standard. Le code du programme ne commence qu'à la fin du fichier (`main` est à la ligne 42 088). Les lignes `# n "fichier"` sont des marqueurs qui indiquent au compilateur d'où vient chaque ligne, pour que ses messages d'erreur citent le bon fichier :

```
# 1 "src/test_pile.cpp"
# 1 "/usr/include/c++/15/chrono" 1 3      ← début du contenu de <chrono>
...
int main() {                               ← ligne 42 088 : le code du TP, sans ses commentaires
    cout << "=== 1. PileTableau<int> ===" << endl;
    PileTableau<int> pt(2);
```

2. Compilation — `test_pile.s`

```
g++ -Wall -Wextra -O2 -Iinclude -S compilation/test_pile.ii -o compilation/test_pile.s
```

Le compilateur proprement dit analyse le C++ (syntaxe, types, templates), l'optimise (`-O2`) et le traduit en **assembleur x86-64**, un texte lisible où chaque ligne correspond à une instruction du processeur. C'est à cette étape que les templates sont instanciés et que les erreurs de type sont détectées.

```
main:
    endbr64
    pushq   %r15
    leaq    _ZSt4cout(%rip), %rdi          ← adresse de std::cout
    ...
    call    _ZN11PileTableauIiE7empilerERKi@PLT ← appel de pt.empiler(i * 10)
```

Les noms de fonctions sont **décorés** (*name mangling*) : `_ZN11PileTableauIiE7empilerERKi` encode `PileTableau<int>::empiler(int const&)`, avec la classe, le paramètre de template et le type des arguments. C'est ce qui permet la surcharge en C++. Le suffixe `@PLT` indique que la fonction est définie ailleurs, dans une bibliothèque partagée.

3. Assemblage — `test_pile.o`

```
g++ -c compilation/test_pile.s -o compilation/test_pile.o
```

L'assembleur traduit chaque instruction en **code machine** binaire. Le résultat est un fichier objet ELF « relocatable » : il contient le code de `test_pile.cpp`, mais **il n'est pas exécutable**, car les adresses des fonctions externes sont encore inconnues. La commande `nm` liste ses symboles :

```
$ nm -C compilation/test_pile.o
                 U PileTableau<int>::empiler(int const&)  ← U : utilisé mais non défini ici
                 U PileTableau<int>::depiler()
                 U PileTableau<char>::PileTableau(int)
0000000000000000 W PileTableau<Point>::empiler(Point const&) ← W : défini ici
```

Le fichier a **61 symboles non définis** (U) : les méthodes de `PileTableau<int>`, `PileListe<string>`... (compilées dans `libpile.so`) et celles de la bibliothèque standard (`std::cout`...). À l'inverse, `PileTableau<Point>` est défini dans le `.o` (W) : `Point` est un type du programme, donc la bibliothèque ne

peut pas le connaître, et `test_pile.cpp` inclut `PileTableau.hpp` pour que le compilateur génère ce code lui-même.

Les bibliothèques — `libpile.a` et `libpile.so`

Les sources `PileTableau.cpp`, `PileListe.cpp` et `Expression.cpp` sont d'abord compilées en fichiers objet (`obj/*.o`), avec l'option `-fPIC` (code indépendant de la position, qui peut être chargé à n'importe quelle adresse en mémoire). Ces trois `.o` sont ensuite regroupés de deux façons :

```
ar rcs compilation/libpile.a obj/PileTableau.o obj/PileListe.o obj/Expression.o # statique
g++ -shared -Wl,-soname,libpile.so obj/*.o -o libpile.so # dynamique
```

	Statique <code>libpile.a</code>	Dynamique <code>libpile.so</code>
Nature	simple archive des <code>.o</code> (<code>ar t</code> les liste)	binaire ELF prêt à être chargé, 369 symboles exportés
Utilisation	le code est copié dans l'exécutable à l'édition de liens	le code est chargé au lancement du programme
Au lancement	le fichier <code>.a</code> n'est plus nécessaire	le fichier <code>.so</code> doit être présent
Mise à jour	il faut relier chaque programme	remplacer le <code>.so</code> suffit
Équivalent Windows	<code>.lib</code>	<code>.dll</code>

4. Édition de liens — les exécutables

L'éditeur de liens (`ld`, appelé par `g++`) associe chaque symbole `U` du `.o` à sa définition, ajoute le code de démarrage du programme et produit un exécutable ELF.

Liaison dynamique — `test_pile`

```
g++ compilation/test_pile.o -Lcompilation -lpile -Wl,-rpath,'$ORIGIN' -o compilation/test_pile
```

`-lpile` cherche `libpile.so` dans le dossier donné par `-L`. L'éditeur de liens vérifie que chaque symbole existe, mais ne copie pas le code : il note seulement que le programme a besoin de `libpile.so`. L'option `-rpath,'$ORIGIN'` indique au système de chercher la bibliothèque dans le dossier de l'exécutable. Au lancement, le chargeur `ld-linux` charge les bibliothèques listées par `ldd` :

```
$ ldd compilation/test_pile
    libpile.so => ../tpl/compilation/libpile.so
    libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6
    libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6
    ...
```

Liaison statique — `test_pile_statique`

```
g++ compilation/test_pile.o compilation/libpile.a -o compilation/test_pile_statique
```

Ici l'éditeur de liens **copie** dans l'exécutable le code des `.o` de l'archive dont il a besoin. Le programme ne dépend plus de `libpile.so` (`ldd` ne la liste pas), mais il est deux fois plus gros : 59 Ko contre 29 Ko. Il reste lié dynamiquement à la bibliothèque standard (`libstdc++`, `libc`).